Inverse Fast Fourier Transform Implementation in MIPS Assembly

Thumu Abhinav

March 1, 2025

1 Introduction

The Inverse Fast Fourier Transform (IFFT) is an efficient algorithm to compute the Inverse Discrete Fourier Transform (IDFT). It is used to reconstruct a signal from its frequency components. This implementation uses the Cooley-Tukey recursive approach, which divides the problem into smaller subproblems by splitting even and odd indexed elements until reaching base cases of size 2. The transform is computed using the butterfly operation:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{2\pi ikn/N} \tag{1}$$

Where:

- $x[n]$ = Reconstructed time-domain signal
- $X[k]$ = Frequency-domain components
- N = Number of points

2 Implementation Details

The MIPS implementation supports transforms of size N=2 and N=4, with key features:

2.1 Memory Organization

- Complex numbers stored as consecutive 32-bit words (real, imaginary)
- Precomputed twiddle factors:
  - `twiddle_n2`: $[1, 0]$ and $[-1, 0]$
  - `twiddle_n4`: $[1, 0]$, $[0, -1]$
- Input array allocated as 32 bytes (8 words) for 4 complex numbers

2.2 Key Implementation Steps

1. **IFFT-N2 Base Case**:

```
ifft_n2:
    lw $t0, 0($s1)  # X[0].real
    lw $t1, 4($s1)  # X[0].imag
    lw $t2, 8($s1)  # X[1].real
    lw $t3, 12($s1) # X[1].imag

    sub $t4, $t0, $t2   # x0.real
    sub $t5, $t1, $t3   # x0.imag
    add $t6, $t0, $t2   # x1.real
    add $t7, $t1, $t3   # x1.imag


    sra $t4, $t4, 1     # X0.real /= 2
    sra $t5, $t5, 1     # X0.imag /= 2
    sra $t6, $t6, 1     # X1.real /= 2
    sra $t7, $t7, 1     # X1.imag /= 2
```

2. **Complex Multiplication**:

```
    mul $s4, $t2, $t9    # Real part calculation
    mul $s5, $t3, $t8
    sub $s4, $s4, $s5    # Final real
    mul $s6, $t2, $t8
    mul $s7, $t3, $t9
    add $s5, $s6, $s7    # Final imag
```

3. **Recursive Call**:

```
    # Compute IFFT2 on even elements
    li $a0, 2
    move $a1, $sp
    jal ifft
```

4. **Result**:

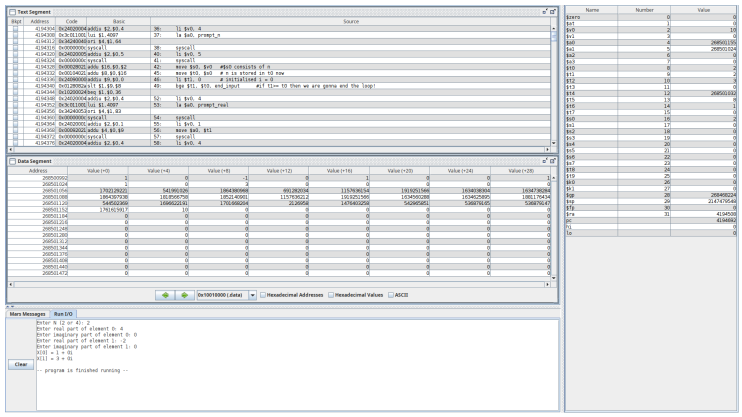

Figure 1: IFFT Computation Results

3 Conclusion

This implementation demonstrates fundamental IFFT principles in low-level assembly while handling:

- Recursive algorithm implementation
- Complex number arithmetic
- MIPS memory and stack management
- System call integration for I/O